

Naive Bayes for Sentiment Analysis

In this assignment you will implement and experiment with various feature engineering techniques in the context of Naive Bayes models for Sentiment classification of movie reviews.

We will use the NB model implemented in sklearn:

https://scikit-learn.org/stable/modules/naive_bayes.html

Write Your Name Here: Kaspian Thomas

1. List item
2. List item

Submission Instructions

1. Click the Save button at the top of the Jupyter Notebook/Google Colab.
2. Please make sure to have entered your name above.
3. Select Cell -> All Output -> Clear. This will clear all the outputs from all cells (but will keep the content of all cells).
4. Select Cell -> Run All. This will run all the cells in order, and will take several minutes.
5. Once you've rerun everything, select File -> Download as (or Print) -> PDF and download a PDF version *SentimentAnalysis.pdf* showing the code and the output of all cells, and save it in the same folder that contains the notebook file *SentimentAnalysis.ipynb*.
6. Look at the PDF file and make sure all your solutions are there, displayed correctly. The PDF is the only thing we will see when grading!
7. Submit **both** your PDF and notebook on Canvas.
8. Verify your Canvas submission contains the correct files by downloading it after posting it on Canvas.

Make sure that you format your solutions to theory questions to show equations properly. We will not grade solutions that are not properly formatted. Jupyter-notebook understands Latex, alternatively you can edit in Word using its Equation editor and submit the PDF as a separate file in Canvas.

Theory (20p)

Theory: J&M Exercise 4.1 (20p)

Assume equal prior probabilities --> ("I always like foreign films")

$$P(pos) = 0.50$$

&

$$P(neg) = 0.50$$

Using given table of

$$P(pos) \rightarrow (0.09, 0.07, 0.29, 0.04, 0.08)$$

$$P(neg) \rightarrow (0.16, 0.06, 0.06, 0.15, 0.11)$$

Using the Bayes Theorem:

$$P(A|B) = [P(B|A) * P(A)]/P(B)$$

$$P(pos) \rightarrow (0.50 * 0.09 * 0.07 * 0.29 * 0.04 * 0.08 \approx 0.0000029232)$$

$$P(neg) \rightarrow (0.50 * 0.16 * 0.06 * 0.06 * 0.15 * 0.11 \approx 0.000004752)$$

Based on the results of the calculations, the sentence is classified as negative (because $P(neg) > P(pos)$). Note: The calculation was done using the Naive Bayes assumptions that assume the words are independent of each other given the class.

Implementation (80p + Bonus 30p)

From documents to feature vectors

This section illustrates the prototypical components of machine learning pipeline for an NLP task, in this case document classification:

1. Read document examples (train, devel, test) from files with a predefined format:
 - assume one document per line, using the format "<label> <text>".
2. Tokenize each document:
 - using a spaCy tokenizer.
3. Feature extractors:
 - so far, just words.
4. Process each document into a feature vector:
 - map document to a dictionary of feature names.
 - map feature names to unique feature IDs.
 - each document is a feature vector, where each feature ID is mapped to a feature value (e.g. word occurrences).

```
import spacy
from spacy.lang.en import English
from scipy import sparse
from sklearn.naive_bayes import MultinomialNB
```

```
import spacy
from spacy.lang.en import English
from scipy import sparse
from sklearn.naive_bayes import MultinomialNB
```

```
# Create spaCy tokenizer.
spacy_nlp = English()
spacy_nlp.add_pipe('sentencizer')
```

```
def spacy_tokenizer(text):
    tokens = spacy_nlp.tokenizer(text)

    return [token.text for token in tokens]
```

```
def read_examples(filename):
    X = []
    Y = []
    with open(filename, mode = 'r', encoding = 'utf-8') as file:
        for line in file:
            [label, text] = line.rstrip().split(' ', maxsplit = 1)
            X.append(text)
            Y.append(label)
    return X, Y
```

```
def word_features(tokens):
    feats = {}
    for word in tokens:
        feat = 'WORD_%s' % word
        if feat in feats:
            feats[feat] += 1
        else:
            feats[feat] = 1
    return feats
```

```
def add_features(feats, new_feats):
    for feat in new_feats:
        if feat in feats:
            feats[feat] += new_feats[feat]
        else:
            feats[feat] = new_feats[feat]
    return feats
```

This function tokenizes the document, runs all the feature extractors on it and assembles the extracted features into a dictionary mapping feature names to feature values. It is important that feature names do not conflict with each other, i.e. different features should have different names. Each document will have its own dictionary of features and their values.

```
def docs2features(trainX, feature_functions, tokenizer):
    examples = []
    count = 0
    for doc in trainX:
        feats = {}

        tokens = tokenizer(doc)

        for func in feature_functions:
            add_features(feats, func(tokens))

        examples.append(feats)
        count += 1

    if count % 100 == 0:
        print('Processed %d examples into features' % len(examples))

    return examples
```

This helper function converts feature names to unique numerical IDs.

```
def create_vocab(examples):
    feature_vocab = {}
    idx = 0
    for example in examples:
        for feat in example:
            if feat not in feature_vocab:
                feature_vocab[feat] = idx
                idx += 1

    return feature_vocab
```

This helper function converts a set of examples from a dictionary of feature names to values representation
to a sparse representation of feature ids to values. This is important because almost all feature values will
be 0 for most documents and it would be wasteful to save all in memory.

```
def features_to_ids(examples, feature_vocab):
    new_examples = sparse.lil_matrix((len(examples), len(feature_vocab)))
    for idx, example in enumerate(examples):
        for feat in example:
            if feat in feature_vocab:
                new_examples[idx, feature_vocab[feat]] = example[feat]

    return new_examples
```

Evaluation pipeline for the Naive Bayes classifier.

```
def train_and_test(trainX, trainY, devX, devY, feature_functions, tokenizer):
    # Pre-process training documents.
    trainX_feat = docs2features(trainX, feature_functions, tokenizer)

    # Create vocabulary from features in training examples.
    feature_vocab = create_vocab(trainX_feat)
    print('Vocabulary size: %d' % len(feature_vocab))

    trainX_ids = features_to_ids(trainX_feat, feature_vocab)

    # Train NB model.
    nb_model = MultinomialNB(alpha = 1.0)
    nb_model.fit(trainX_ids, trainY)

    # Pre-process test documents.
    devX_feat = docs2features(devX, feature_functions, tokenizer)
    devX_ids = features_to_ids(devX_feat, feature_vocab)

    # Test NB model.
    print('Accuracy: %.3f' % nb_model.score(devX_ids, devY))
```

```
# For Colab only
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import os

# For Colab
datapath = '/content/drive/MyDrive/HW3/data'

# For Notebook
# datapath = '../data'

train_file = os.path.join(datapath, 'imdb_sentiment_train.txt')
trainX, trainY = read_examples(train_file)

dev_file = os.path.join(datapath, 'imdb_sentiment_dev.txt')
devX, devY = read_examples(dev_file)

# Specify features to use.
features = [word_features]

# Evaluate NB model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)
```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 28692
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.779
```

▼ Feature engineering

[10p] Evaluate NB model performance when using only alpha tokens. This can be done by changing the tokenizer function.

```
def spacy_tokenizer1(text):
    tokens = spacy_nlp.tokenizer(text)

    # Use a list comprehension to keep only tokens where the text is strictly letters
    filtered_tokens = [token.text for token in tokens if token.text.isalpha()]

    return filtered_tokens

# Evaluate NB model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer1)
```

```
Processed 100 examples into features
Processed 200 examples into features
```

```

Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 25054
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.785

```

[10p] Same as above, but lowercase all tokens before using as features.

```

def spacy_tokenizer2(text):
    tokens = spacy_nlp.tokenizer(text)

    # Use a list comprehension to filter for letters AND convert to lowercase
    filtered_tokens = [token.text.lower() for token in tokens if token.text.isalpha()]

    return filtered_tokens

# Evaluate NB model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer2)

```

```

Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 21708
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.784

```

[15p] Same as above, but lowercase only tokens that appear at the beginning of sentences.

```
def spacy_tokenizer3(text):
    doc = spacy_nlp(text)

    tokens = []

    # Iterate through each sentence
    for sent in doc.sents:
        # Iterate through each token in the sentence
        for i, token in enumerate(sent):
            # Only keep the token if it is made up of letters
            if token.text.isalpha():
                # If it's the very first token in the sentence (index 0), lowercase it
                if i == 0:
                    tokens.append(token.text.lower())
                # Otherwise, keep its original case
            else:
                tokens.append(token.text)

    return tokens

# Evaluate NB model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer3)
```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 25054
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.784
```

[15p] Use `spacy_tokenizer2` (only alpha tokens, lowered all) and display the top 10 most frequent tokens in the vocabulary, as a list of tuples (token, frequency).

```
# First, count token occurrences across all examples, where features are still strings.
def create_feature_counts(examples):
    feature_counts = {}

    # Iterate through every document's feature dictionary in the dataset
    for example in examples:
        # Iterate through every feature and its frequency in the current document
        for feat, count in example.items():
            # Add the count to our global feature_counts dictionary
            feature_counts[feat] = feature_counts.get(feat, 0) + count

    return feature_counts

# Create features for all training examples, compute feature counts
def fcounts_from_train(trainX, feature_functions, tokenizer):
    # Pre-process training documents.
    trainX_feat = docs2features(trainX, feature_functions, tokenizer)

    # Create vocabulary from features in training examples.
```

```

feature_counts = create_feature_counts(trainX_feat)
print('Vocabulary size: %d' % len(feature_counts))

return feature_counts

# Return a list of the top K most frequent tokens in the vocabulary.
def topK_tokens(vocab, k):
    # vocab.items() give a list of tuples
    # key=lambda x: x[1] tells Python to sort based on the second item in the tuple (the count)
    # reverse=True sorts from highest to lowest
    # [:k] slices the list to return only the top 'k' results

    top_k = sorted(vocab.items(), key=lambda x: x[1], reverse=True)[:k]

    return top_k

vocab = fcounts_from_train(trainX, features, spacy_tokenizer2)
stop_words = topK_tokens(vocab, 20)
for item in stop_words:
    print(item)

```

```

Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 21708
('WORD_the', 20105)
('WORD_and', 9876)
('WORD_a', 9749)
('WORD_of', 9029)
('WORD_to', 8182)
('WORD_is', 6605)
('WORD_in', 5598)
('WORD_it', 5387)
('WORD_i', 4773)
('WORD_this', 4448)
('WORD_that', 4252)
('WORD_was', 2961)
('WORD_with', 2693)
('WORD_as', 2682)
('WORD_movie', 2574)
('WORD_for', 2556)
('WORD_but', 2435)
('WORD_film', 2359)
('WORD_you', 2066)
('WORD_on', 2061)

```

[15p] Evaluate NB model performance when ignoring the top 20 stop words. Use spacy_tokenizer2 (only alpha tokens, lowered all).

```

# Evaluation pipeline for the Naive Bayes classifier.

def train_and_test(trainX, trainY, devX, devY, feature_functions, tokenizer):
    # Pre-process training documents.
    trainX_feat = docs2features(trainX, feature_functions, tokenizer)

    # Create vocabulary from features in training examples.
    feature_counts = create_feature_counts(trainX_feat)

    # FIX: Use feature_counts instead of 'vocab'
    stop_words = topK_tokens(feature_counts, 20)

    # Remove from each example features that appear in the stop words list.
    # 1. Extract just the feature names into a set for fast lookup
    stop_word_names = set([word for word, count in stop_words])

    # 2. Iterate through every document's feature dictionary
    for example in trainX_feat:
        # Create a list of keys to avoid 'dictionary changed size during iteration' error

```

```

    for feat in list(example.keys()):
        if feat in stop_word_names:
            del example[feat]

# Create vocabulary from features in training examples.
feature_vocab = create_vocab(trainX_feat)
print('Vocabulary size: %d' % len(feature_vocab))

trainX_ids = features_to_ids(trainX_feat, feature_vocab)

# Train NB model.
nb_model = MultinomialNB(alpha = 1.0)
nb_model.fit(trainX_ids, trainY)

# Pre-process test documents.
devX_feat = docs2features(devX, feature_functions, tokenizer)
devX_ids = features_to_ids(devX_feat, feature_vocab)

# Test NB model.
print('Accuracy: %.3f' % nb_model.score(devX_ids, devY))

# Evaluate NB model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer2)

```

```

Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 21688
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.798

```

[5p] Evaluate NB model performance when ignoring words that appear less than 5 times. Use spacy_tokenizer2 (only alpha tokens, lowered all).

```

def train_and_test(trainX, trainY, devX, devY, feature_functions, tokenizer):
    # 1. Pre-process training documents into features
    trainX_feat = docs2features(trainX, feature_functions, tokenizer)

    # 2. Count occurrences of every feature across the whole training set
    feature_counts = create_feature_counts(trainX_feat)

    # 3. Remove words appearing less than 5 times
    for example in trainX_feat:
        for feat in list(example.keys()):
            if feature_counts.get(feat, 0) < 5:
                del example[feat]

    # 4. Create the vocabulary from the remaining filtered features
    feature_vocab = create_vocab(trainX_feat)
    print('Vocabulary size after filtering: %d' % len(feature_vocab))

    # 5. Convert to sparse ID representation

```



```

trainX_ids = features_to_ids(trainX_feat, feature_vocab)

# 6. Train the Multinomial Naive Bayes model
nb_model = MultinomialNB(alpha = 1.0)
nb_model.fit(trainX_ids, trainY)

# 7. Pre-process and test on development documents
devX_feat = docs2features(devX, feature_functions, tokenizer)
devX_ids = features_to_ids(devX_feat, feature_vocab)

# 8. Output Final Accuracy
print('Accuracy: %.3f' % nb_model.score(devX_ids, devY))

# Evaluate the final NB model configuration
features = [word_features]
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer2)

Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size after filtering: 5573
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.791

```

▼ Analysis (10p)

Include an analysis of the results that you obtained in the experiments above. You can take advantage of the Jupyter Notebook markdown language, which can also process Latex and HTML, to format your report so that it looks professional.

Based on the experiments, the predormance of the Multinomial Naive Bayes model using multiple different tokenization and filtering techqniques are as follows:

Experiment --> Filtering --> Accuracy --> Vocabulary Size

- Baseline(spacy_tokenizer) --> 77.7% --> 28,692
- AlphaOnly(spacy_tokenizer1) --> 78.5% --> 25,054
- Lowercase(spacy_tokenizer2) --> 78.4% --> 21,708
- Sent-Start Lower(spacy_tokenizer3) --> 78.4 --> 25,054
- Stop Words --> 78.9% --> 21,688
- Freq Filtering --> 79.1% --> 5,573

Removing punctuation and numbers increased accuracy from 0.779 to 0.785 and reduced the vocabulary size by over 3,600 tokens. Converting all alpha tokens to lowercase also reduced the vocab size down to 21,708. Removing the more frequent tokens gave a higher accuracy of 0.789. Filtering rare words that apperead less than 5 times reduced the vocabulary from 21,708 to 5,573, but the accuracy dropped slightly to 0.791.

✓ [Bonus] Binary Multinomial Bayes (20p)

*Optional

Write code for transforming documents to features such that features are Boolean and only represent whether a word occurred in a document, as in the Binary Multinomial Naive Bayes discussed in class. Evaluate the Naive Bayes model with this feature representation, using spacy_tokenizer2 (only alpha tokens, lowered all).

Double-click (or enter) to edit